Week 08: Scope, Work Breakdown and Estimation

R&D and Project Management in Computer Science

R&D and Project Management in Computer Science (11117BLG002). Prof. Dr. Utku Kose. Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye. Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India). ORCID 0000-0002-9652-6415. E-mail: utkukose@sdu.edu.tr, utku.kose@und.edu, ukose@up.edu.mx, utkukose@gmail.com. This course is updated in line with current developments in the field. First version: September 2026. Last update: October 2026.

Overview

A plan is only as good as its estimates. This week covers the definition of scope, the work breakdown structure and the main estimation methods, from three-point estimates to parametric models, together with the biases that make estimates optimistic [1, 2, 3].

Learning outcomes

By the end of the week, students are expected to build a work breakdown structure that covers the whole scope. They should be able to compute three-point estimates and their uncertainty and to apply a basic parametric model. Finally, they should be able to explain the planning fallacy, reference class forecasting and Brooks's law.

Lecture

Scope and the work breakdown structure

Project management standards describe planning as the definition of what will and will not be done, followed by the organisation of the work [1, 4]. The work breakdown structure decomposes the total scope into deliverable-oriented components down to work packages that can be estimated and assigned. A widely used rule requires that the structure cover all of the work in the scope and nothing outside it, so that the sum of the parts equals the whole. In research projects, the structure usually mirrors the work packages of the proposal, with management, dissemination and data management as packages of their own.

Estimation methods

Estimates can be made by analogy with similar past work, by expert judgement, by parametric models or by combining optimistic, most likely and pessimistic values. Jørgensen and Shepperd systematically reviewed research on software cost estimation and classified the approaches that studies examined, from formal models to expert judgement [5]. The three-point estimate of

PERT was introduced for a research and development programme [6]. It weights the most likely value four times: The mean is (a + 4m + b) / 6 and the standard deviation (b - a) / 6. When the pessimistic value lies far above the most likely one, as is common in research, the mean exceeds the most likely value, as Figure 8.1 shows. Boehm's Constructive Cost Model, COCOMO, estimates effort from program size with coefficients calibrated on past projects [2]. In its basic form, effort in person-months is a times thousands of lines of code to the power b. The values of a and b depend on the type of project.

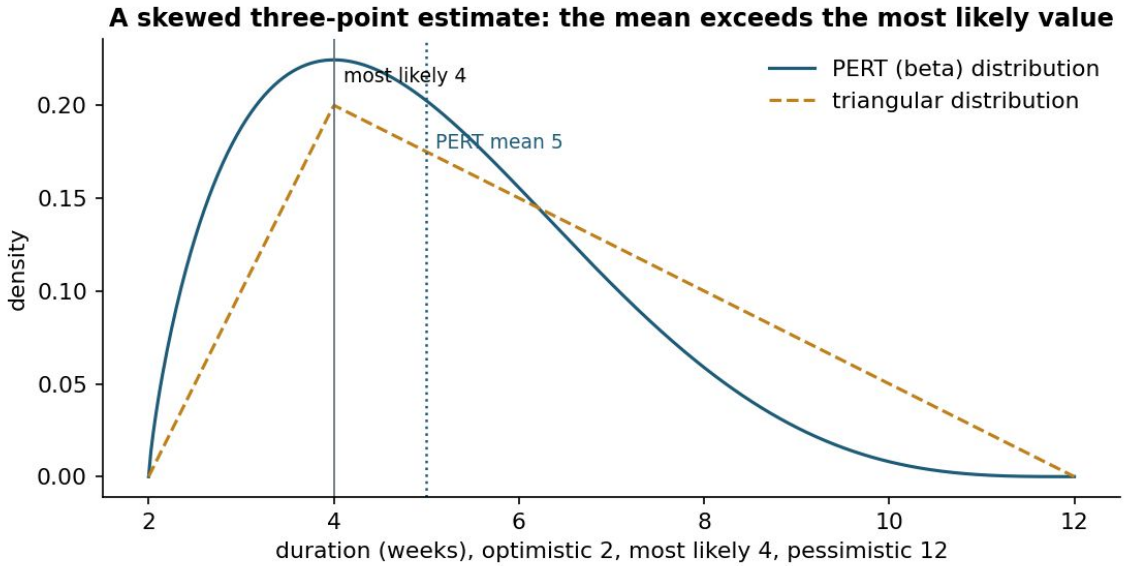


Figure 8.1. A three-point estimate with a long pessimistic tail: Under both the PERT and the triangular distribution, the expected duration exceeds the most likely one [6].

Why estimates are optimistic

Buehler, Griffin and Ross demonstrated the planning fallacy [7]. People predict that their own tasks will be completed sooner than they actually are, even when they know that similar tasks took longer. Flyvbjerg argued that such optimism bias and strategic misrepresentation explain much of the overrun of large projects [3]. He proposed reference class forecasting, which bases estimates on the distribution of outcomes in a class of similar past projects rather than on the details of the plan. For proposals, a simple version is to compare planned effort with the actual effort of earlier projects of the same group.

People and time

Brooks observed that adding people to a late software project makes it later [8]. New members need training, and the communication effort grows with the number of pairs of people, n(n - 1) / 2. Person-months are therefore not interchangeable with months. In research teams, a doctoral student who joins in the second year cannot compensate for the first year, and the plan must reflect when each person is available. The interactive lab combines a work breakdown with three-point estimates, a basic COCOMO calculator and a counter of communication channels.

> ***Pause and reflect.*** *Compare the estimate of your last thesis chapter or project task with the time it actually took. What would a reference class have told you?*

Hands-on components

Interactive lab: Part A reads an indented work breakdown with optimistic, most likely and pessimistic estimates and sums them. Part B applies the basic COCOMO model. Part C counts communication channels in a team [2, 6, 8]. Link: https://utkukose.github.io/rd-project-management-NB-lecture/weeks/week-08/lab.html

Colab notebook: It computes three-point estimates, compares their normal approximation with a Monte Carlo sum and applies the basic COCOMO model [2, 6]. Link: https://colab.research.google.com/github/utkukose/rd-project-management-NB-lecture/blob/main/weeks/week-08/NB08_scope_estimation.ipynb

Reflection prompts

1. Which element of your work breakdown has the widest range between optimistic and pessimistic estimates, and why?
2. What reference class would you use for your project, and where could you find its data?
3. How does the availability of each team member over time constrain your plan?

Weekly task and submission

What to do. Build the work breakdown structure of your project and estimate it. First, break the work packages into tasks and give each task an optimistic, a most likely and a pessimistic duration. Second, report the PERT mean and standard deviation of the total. Add an estimate that you would meet with 85 percent probability, which under a normal approximation is about the mean plus one standard deviation. Third, compare the total with a reference class of at least three earlier projects of similar scope.

Where to look. Lecture: sections "Scope and the work breakdown structure", "Estimation methods" and "Why estimates are optimistic". Interactive lab: Part A, which sums three-point estimates from an indented work breakdown. Colab notebook: Section 1 with Exercise 1 for the PERT estimates.

Example. A data collection task could take 4, 6 or 10 weeks. Its PERT mean is (4 + 4 x 6 + 10) / 6, about 6.3 weeks, and its standard deviation is (10 - 4) / 6 = 1 week. If three earlier theses needed 30 to 50 percent more time than planned, the reference class suggests a larger buffer.

What to submit. The work breakdown structure with a table of the estimates, the PERT totals and the 85 percent estimate. Add the comparison with the reference class and the Colab notebook with Exercises 1 and 2 completed.

The weekly task supports self-learning and builds a personal portfolio. In an active semester, students who follow the course with the instructor can send their work for evaluation to utkukose@sdu.edu.tr or utkukose@gmail.com. The e-mail contains the items that the task lists under What to submit. It also includes the learning log exported from the interactive lab (tab: Reflect and submit).

References

[1] Project Management Institute (2021). A Guide to the Project Management Body of Knowledge (PMBOK Guide) and The Standard for Project Management (7th ed.). Project Management Institute.

[2] Boehm, B. W. (1981). Software Engineering Economics. Prentice Hall.

[3] Flyvbjerg, B. (2006). From Nobel Prize to project management: Getting risks right. Project Management Journal, 37(3), 5-15. https://doi.org/10.1177/875697280603700302

[4] ISO (2020). ISO 21502:2020 Project, programme and portfolio management: Guidance on project management. International Organization for Standardization.

[5] Jørgensen, M., & Shepperd, M. (2007). A systematic review of software development cost estimation studies. IEEE Transactions on Software Engineering, 33(1), 33-53. https://doi.org/10.1109/TSE.2007.256943

[6] Malcolm, D. G., Roseboom, J. H., Clark, C. E., & Fazar, W. (1959). Application of a technique for research and development program evaluation. Operations Research, 7(5), 646-669. https://doi.org/10.1287/opre.7.5.646

[7] Buehler, R., Griffin, D., & Ross, M. (1994). Exploring the planning fallacy: Why people underestimate their task completion times. Journal of Personality and Social Psychology, 67(3), 366-381. https://doi.org/10.1037/0022-3514.67.3.366

[8] Brooks, F. P. (1975). The Mythical Man-Month: Essays on Software Engineering. Addison-Wesley.